Parul University

FACULTY OF ENGINEERING & TECHNOLOGY

BACHELOR OF TECHNOLOGY

(303105259)

Competitive Coding

4st SEMESTER

COMPUTER SCIENCE & ENGINEERING DEPARTMENT

Laboratory Manual

PREFACE

It gives us immense pleasure to present the first edition of ***(303105259) Competitive Coding*** for the B.Tech. 1st year students for PARUL UNIVERSITY.

The Fundamental of Programming theory and laboratory courses at **PARUL UNIVERSITY, WAGHODIA, VADODARA** are designed in such a way that students develop the basic understanding of the subject in the theory classes and then try their hands on the computer learnt during the theoretical sessions.

This book is emphatically not focused on *Competitive Coding* "The only". Understanding the fundamental ideals, principals, and techniques is the essence of a good programmer. Only well-designed code has a chance of becoming part of a correct, reliable, and maintainable system. Also, "the fundamentals" are what last: they will still be essential after today's language and tools have evolved or been replaced.

We acknowledge the authors and publishers of all the books which we have consulted whiledeveloping this Practical book. Hopefully this *(303105259) Competitive Coding* will serve the purpose for which it has been developed.

Instructions to students

1. Every student should obtain a copy of laboratory Manual.
2. Dress Code: Students must come to the laboratory wearing.
   i. Trousers,
   ii. half-sleeve tops and
   iii. Leather shoes. Half pants, loosely hanging garments and slippers are not allowed.
3. To avoid injury, the student must take the permission of the laboratory staff before handling anymachine.
4. Students must ensure that their work areas are clean and dry to avoid slipping.
5. Do not eat or drink in the laboratory.
6. Do not remove anything from the computer laboratory without permission.
7. Do not touch, connect or disconnect any plug or cable without your lecturer/laboratorytechnician's permission.
8. All students need to perform the practical/program.

CERTIFICATE

This is to certify that

*Mr./Ms*_____*with enrolment no.*

_____*has successfully completed his/her laboratory experiments*

in the(303105259) Competitive Coding From the department of

..... during the academic year .....

Date of Submission:.....

Staff In charge:.....

Head of Department:.....

INDEX

Class: 4st Sem

SUBJECT: *Competitive Coding*

A.Y.20_ -20_____

SUBJECT CODE (303105259)

sr. no	Experiment Title	Page no	Date of Performance	Date of Submission	Marks out of 10	Sign
	Practical					
1	Write a program for implementing a MINSTACK which should support operations like push, pop, overflow, underflow, display 1. Construct a stack of N-capacity 2. Push elements 3. Pop elements 4. Top element 5. Retrieve the min element from the stack .					
2	Write a program to deal with real-world situations where Stack data structure is widely used Evaluation of expression: Stacks are used to evaluate expressions, especially in languages that use postfix or prefix notation. Operators and operands are pushed onto the stack, and operations are performed based on the LIFO principle.					
3	Write a program for finding NGENEXT GREATER ELEMENT from an array					
4	Write a program to design a circular queue(k) which Should implement the below functions a. Enqueue b. Dequeue c. Front d. Rear					
5	Write a Program for an infix expression, and convert it to postfix notation. Use a queue to implement the Shunting Yard Algorithm for expression conversion.					

6	Write a Program for finding the Product of the three largest Distinct Elements. Use a Priority Queue to efficiently find and remove the largest elements.					
7	Write a Program to Merge two linked lists(sorted)					
8	Write a Program to find the Merge point of two linked lists(sorted).					
9	Write a Program to Swap Nodes pairwise.					
10	Write a Program for Building a Function ISVALID to VALIDATE BST.					
11	Write a Program to Build BST.					
12	Write a Program to determine the depth of a given Tree by Implementing MAXDEPTH.					
13	Write a Program to Understand and implement Tree traversals i.e. Pre-Order Post-Order, In-Order.					
14	Write a Program to perform Boundary Traversal on BST.					
15	Write a program for Lowest Common Ancestors.					
16	Write a Program to verify and validate mirrored trees or not.					
17	Write a Program for a basic hash function in a programming language of your choice. Demonstrate its usage to store and retrieve key-value pairs.					

18	Implement a hash table using separate chaining for collision handling. Perform operations like insertion, deletion, and search on the hash table.					
19	Write a Program to Implement Two sums using HASHMAP.					
20	Write a Program to Implement Search, insert, and Remove in Trie.					
21	Write a Program to Implement Huffman coding.					
22	Write a Program to find Distinct substrings in a string.					
23	Write a Program to find The No of Words in a Trie.					
24	Write a Program to view a tree from left View.					
25	Write a Program to Traverse a Tree using Level Order Traversal.					

PRACTICAL-01:

AIM : Write a program for implementing a MINSTACK which should support operations like push, pop, overflow, underflow, display

1. **Construct a stack of N-capacity**
2. **Push elements**
3. **Pop elements**
4. **Top element**
5. **Retrieve the min element from the stack.**

Problem Statement:

Design a MinStack data structure that supports the following operations:

1. **Push**: Add an element to the top of the stack.
2. **Pop**: Remove the element from the top of the stack.
3. **Top**: Retrieve the top element of the stack without removing it.
4. **GetMin**: Retrieve the minimum element in the stack.
5. **Display**: Display all elements in the stack.
6. **Overflow**: Check if the stack is full (if a capacity is defined).
7. **Underflow**: Check if the stack is empty.

The stack should have a fixed capacity N, and it should handle overflow and underflow conditions appropriately.

Algorithms:

1. **Initialize the Stack**:
   - Create two stacks: one for storing the actual elements (stack) and another for storing the minimum elements (min_stack).
   - Define the capacity N of the stack.
2. **Push Operation**:
   - Check if the stack is full (overflow condition). If full, display an error message.
   - Push the element onto the stack.
   - If the min_stack is empty or the new element is less than or equal to the top of the min_stack, push the element onto the min_stack.

3. **Pop Operation**:
   - Check ifthe stack is empty(underflow condition). If empty, display an error message.
   - Pop the top element from the stack.
   - If the popped element is equal to the top ofthe min_stack, pop the top element from the min_stack.
4. **Top Operation**:
   - Return the top element of the stack without removing it.
5. **GetMin Operation**:
   - Return the top element of the min_stack, which is the minimum element in the stack.
6. **Display Operation**:
   - Print all elements in the stack.
7. **Overflow Check**:
   - Return True if the stack is full, otherwise False.
8. **Underflow Check**:
   - Return True if the stack is empty, otherwise False.

Code:

```
import java.util.Stack;

class MinStack {

private Stack<Integer> mainStack;

private Stack<Integer> minStack;


public MinStack() {

mainStack = new Stack<>();

minStack = new Stack<>();

}

public void push(int val) {

mainStack.push(val);

if (minStack.isEmpty() || val <= minStack.peek()) {

minStack.push(val);

}

}

public void pop() {

if (mainStack.peek().equals(minStack.peek())) {

minStack.pop();

}
```

```
  mainStack.pop();

public int top() {
  return mainStack.peek();
  }
  public int getMin() {
  return minStack.peek();
  }
  // Main method for testing (required for online compilers)
  public static void main(String[] args) {
  MinStack stack = new MinStack();
  stack.push(5);
  stack.push(3);
  stack.push(7);
  stack.push(3);

  System.out.println("Top: " + stack.top());      // 3
  System.out.println("Min: " + stack.getMin());  // 3

  stack.pop();
  System.out.println("Min after pop: " + stack.getMin()); // 3

  stack.pop();
  System.out.println("Min after pop: " + stack.getMin()); // 3

  stack.pop();
  System.out.println("Top: " + stack.top());      // 5
  System.out.println("Min: " + stack.getMin());  // 5
  }
  }
```

OUTPUT:

```
Top: 3
Min: 3
Min after pop: 3
Min after pop: 3
Top: 5
Min: 5

=== Code Execution Successful ===
```

PRACTICAL-02:

AIM : Write a program to deal with real-world situations where Stack data structure is widely used Evaluation of expression: Stacks are used to evaluate expressions, especially in languages that use postfix or prefix notation. Operators and operands are pushed onto the stack, and operations are performed based on the LIFO principle.

Problem Statement:

In many real-world applications, such as calculators or compilers, expressions need to be evaluated. One common way to evaluate expressions is by using the stack data structure, especially for expressions written in postfix (Reverse Polish Notation) or prefix notation. The stack's Last-In-First-Out (LIFO) property makes it ideal for this purpose.

Your task is to write a Python program that evaluates a postfix expression using a stack. The program should handle basic arithmetic operations like addition (+), subtraction (-), multiplication (*), and division (/).

Algorithm:

1. **Initialize an empty stack** to store operands.
2. **Iterate through each character** in the postfix expression:
   - If the character is an **operand** (number), push it onto the stack.
   - If the character is an **operator** (+, -, *, /):
     - Pop the top two elements from the stack.
     - Perform the operation on the two popped elements.
     - Push the result back onto the stack.
3. After processing all characters, the final result will be the only element left in the stack.
4. **Return the final result**.

CODE:

```
import java.util.Stack;

class Solution {

public int evalRPN(String[] tokens) {



Stack<Integer> stack = new Stack<>();

for (String token : tokens) {

if (token.equals("+") || token.equals("-") ||
```

```
token.equals("*") || token.equals("/")) {

int b = stack.pop();
int a = stack.pop();

switch (token) {
case "+":
stack.push(a + b);
break;
case "-":
stack.push(a - b);
break;
case "*":
stack.push(a * b);
break;

                    case "/":
stack.push(a / b); // integer division
break;
}
} else {
stack.push(Integer.parseInt(token));
}
}
return stack.peek();
}

// Main method for online compiler testing
public static void main(String[] args) {
```

```
Solution sol = new Solution();

String[] tokens1 = {"2", "1", "+", "3", "*"};
System.out.print("tokens1: ");
System.out.println(sol.evalRPN(tokens1)); // Output: 9

String[] tokens2 = {"4", "13", "5", "/", "+"};
System.out.print("tokens2: ");
System.out.println(sol.evalRPN(tokens2)); // Output: 6

String[] tokens3 = {"10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"};
System.out.print("tokens3: ");
System.out.println(sol.evalRPN(tokens3)); // Output: 22
}
}
```

OUTPUT:

```
tokens1: 9
tokens2: 6
tokens3: 22

=== Code Execution Successful ===
```

PRACTICAL-03:

AIM : Write a program for finding NGENEXT GREATER ELEMENT from an array.

Problem Statement:

Given an array of integers, the Next Greater Element (NGE) for each element is the first greater element to its right. If no such element exists, the NGE is considered to be -1. Write a program to find the NGE for each element in the array.

Algorithm:

1. **Initialize** a stack and a result array.
2. **Iterate** through the array from the end to the start.
3. For each element:
   - **Pop** elements from the stack until the top of the stack is greater than the current element or the stack is empty.
   - If the stack is empty, the NGE for the current element is -1.
   - Otherwise, the NGE is the top element of the stack.
   - **Push** the current element onto the stack.
4. **Reverse** the result array to match the original order of the input array.
5. **Return** the result array.

CODE:

```
import java.util.*;


class Solution {
public int[] nextGreaterElement(int[] nums1, int[] nums2) {


Map<Integer, Integer> map = new HashMap<>();

Stack<Integer> stack = new Stack<>();


// Traverse nums2 to find next greater elements

for (int num : nums2) {

while (!stack.isEmpty() && stack.peek() < num) {

map.put(stack.pop(), num);

}
```

```
stack.push(num);
  }

  // Remaining elements have no next greater element
  while (!stack.isEmpty()) {
  map.put(stack.pop(), -1);
  }

  // Build result for nums1
  int[] result = new int[nums1.length];
  for (int i = 0; i < nums1.length; i++) {
  result[i] = map.get(nums1[i]);
  }

  return result;
  }
// Main method for online compiler testing
public static void main(String[] args) {
Solution sol = new Solution();
int[] nums1 = {4, 1, 2};
int[] nums2 = {1, 3, 4, 2};
int[] result = sol.nextGreaterElement(nums1, nums2);
System.out.println("Next Greater Elements:");
for (int val : result) {
  System.out.print(val + " ");
  }
  }
  }
```

OUTPUT:

```
Next Greater Elements:
-1 3 -1
=== Code Execution Successful ===
```

PRACTICAL-04:

AIM : Write a program to design a circular queue(k) which Should implement the below functions

a. Enqueue
b. Dequeue
c. Front
d. Rear

Problem Statement:

Design a circular queue data structure that supports the following operations:

1. **Enqueue**: Insert an element into the queue. If the queue is full, return an error or indicate that the operation cannot be performed.
2. **Dequeue**: Remove an element from the queue. If the queue is empty, return an error or indicate that the operation cannot be performed.
3. **Front**: Get the front item from the queue. If the queue is empty, return an error or indicate that the operation cannot be performed.
4. **Rear**: Get the last item from the queue. If the queue is empty, return an error or indicate that the operation cannot be performed.

The circular queue should efficiently utilize the space and handle wrap-around conditions.

Algorithm:

1. Initialize Circular Queue

1. Start
2. Create an array `queue[k]` of size **k**
3. Set `front = -1`
4. Set `rear = -1`

2. Enqueue Operation (Insert Element)

1. Check if queue is full
   - If `(rear + 1) % k == front`
   - Print **"Queue Overflow"** and stop
2. If `front == -1`
   - Set `front = 0` and `rear = 0`
3. Else
   - Set `rear = (rear + 1) % k`
4. Insert element at `queue[rear]`
5. End

3. Dequeue Operation (Delete Element)

1. Check if queue is empty
   - If `front == -1`

- Print **"Queue Underflow"** and stop

2. Store `queue[front]` in variable `item`
3. If `front == rear`
   - Set `front = -1`
   - Set `rear = -1`
4. Else
   - Set `front = (front + 1) % k`
5. Return `item`

4. Front Operation (Get First Element)

1. If `front == -1`
   - Print **"Queue is Empty"**
2. Else
   - Return `queue[front]`

5. Rear Operation (Get Last Element)

1. If `rear == -1`
   - Print **"Queue is Empty"**
2. Else
   - Return `queue[rear]`

CODE:

```
import java.util.*;


class MyCircularQueue {


    private int[] queue;

    private int front;

    private int rear;

    private int size;

    private int capacity;


    // Constructor

    public MyCircularQueue(int k) {
```

```
    {
        capacity = k;
        queue = new int[k];
        front = 0;
        rear = -1;
        size = 0;
    }

    // Enqueue operation
    public boolean enQueue(int value) {
        if (isFull()) {
            return false;
        }
        rear = (rear + 1) % capacity;
        queue[rear] = value;
        size++;
        return true;
    }

    // Dequeue operation
    public boolean deQueue() {
        if (isEmpty()) {
            return false;
        }
        front = (front + 1) % capacity;
        size--;
        return true;
    }

    // Get Front element
    public int Front() {
```

```
}

    if (isEmpty()) {
        return -1;
    }
    return queue[front];
}

// Get Rear element
public int Rear() {
    if (isEmpty()) {
        return -1;
    }
    return queue[rear];
}

// Check if queue is empty
public boolean isEmpty() {
    return size == 0;
}

// Check if queue is full
public boolean isFull() {
    return size == capacity;
}

// Main method
public static void main(String[] args) {
MyCircularQueue cq = new MyCircularQueue(5);
System.out.println(cq.enQueue(10));
```

```
}

        System.out.println(cq.enQueue(20));
        System.out.println(cq.enQueue(30));
        System.out.println(cq.enQueue(40));
        System.out.println(cq.enQueue(50));
        System.out.println(cq.enQueue(60)); // queue full

        System.out.println("Front: " + cq.Front());
        System.out.println("Rear: " + cq.Rear());

        cq.deQueue();
        cq.deQueue();

        System.out.println("Front after dequeue: " + cq.Front());

        cq.enQueue(60);
        cq.enQueue(70);

        System.out.println("Rear after enqueue: " + cq.Rear());
    }
}
```

OUTPUT:

```
true
true
true
true
true
false
Front: 10
Rear: 50
Front after dequeue: 30
Rear after enqueue: 70

=== Code Execution Successful ===
```

PRACTICAL-05:

AIM : Write a Program for an infix expression, and convert it to postfix notation. Use a queue to implement the Shunting Yard Algorithm for expression conversion.

Problem Statement:

Write a Python program that takes an infix expression as input and converts it to postfix notation using the Shunting Yard Algorithm. The Shunting Yard Algorithm should be implemented using a queue to manage the output and a stack to manage the operators.

Algorithm:

1. **Initialize**:
   - A stack to hold operators.
   - A queue to hold the output (postfix expression).
   - A dictionary to define operator precedence.
2. **Tokenize the Input**:
   - Split the input string into tokens (numbers, operators, parentheses).
3. **Process Each Token**:
   - If the token is a number, add it to the output queue.
   - If the token is an operator:
     - While there is an operator on the top of the stack with higher or equal precedence, pop it and add it to the output queue.
     - Push the current operator onto the stack.
   - If the token is a left parenthesis (, push it onto the stack.
   - If the token is a right parenthesis):
     - Pop from the stack and add to the output queue until a left parenthesis is encountered.
     - Pop the left parenthesis from the stack and discard it. **Empty the Stack**:
   - After all tokens are processed, pop any remaining operators from the stack and add them to the output queue.
4. **Output the Postfix Expression**:
   - Convert the queue to a string to represent the postfix expression.

CODE:

```
import java.util.*;

public class Main {

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.println("Enter Infix Expression:");
String input = sc.nextLine();

Solution sol = new Solution();
String result = sol.infixToPostfix(input);

System.out.println("Postfix Expression: " + result);
}
}

class Solution {

public String infixToPostfix(String s) {

// Queue for output
Queue<String> output = new LinkedList<>();

// Stack for operators
```

```
Stack<Character> operators = new Stack<>();

// Remove spaces
s = s.replaceAll("\\s+", "");

for (int i = 0; i < s.length(); i++) {
char ch = s.charAt(i);

// If operand (letter or digit)
if (Character.isLetterOrDigit(ch)) {
output.add(String.valueOf(ch));
}

// If '(' push to stack
else if (ch == '(') {
operators.push(ch);
}

// If ')'
else if (ch == ')') {
while (!operators.isEmpty() && operators.peek() != '(') {
output.add(String.valueOf(operators.pop()));
}
operators.pop(); // Remove '('
}

// If operator
else {
while (!operators.isEmpty() &&
```

```
precedence(operators.peek()) >= precedence(ch)) {
output.add(String.valueOf(operators.pop()));
}
operators.push(ch);
}
}


// Pop remaining operators
while (!operators.isEmpty()) {
output.add(String.valueOf(operators.pop()));
}

// Convert queue to string
StringBuilder result = new StringBuilder();
while (!output.isEmpty()) {
result.append(output.poll());
}

return result.toString();
}

// Precedence function
private int precedence(char ch) {
if (ch == '+' || ch == '-') return 1;
if (ch == '*' || ch == '/') return 2;
if (ch == '^') return 3;
return 0;
}}
```

OUTPUT:

```
Enter Infix Expression:
(a/b)*c/d+e-f
Postfix Expression: ab/c*d/e+f-


...Program finished with exit code 0
Press ENTER to exit console.
```

```
Enter Infix Expression:
(3*8)/4+2(2-4)
Postfix Expression: 38*4/224-+


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-06:

AIM : Write a Program for finding the Product of the three largest Distinct Elements. Use a Priority Queue to efficiently find and remove the largest elements.

Problem Statement:

Given an array of integers, find the product of the three largest distinct elements in the array. If there are fewer than three distinct elements, return -1. Use a priority queue (heap) to efficiently find and remove the largest elements.

Algorithm:

1. **Input**: An array of integers.
2. **Output**: The product of the three largest distinct elements or -1 if there are fewer than three distinct elements.
3. **Steps**:
   - Use a max-heap (priority queue) to efficiently find and remove the largest elements.
   - Insert all elements into the heap.
   - Extract the top three distinct elements from the heap. ○ If fewer than three distinct elements are found, return -1.
   - Otherwise, return the product of the three largest distinct elements.

CODE:

```
import java.util.*;

class ProductOfThreeLargestDistinct {

// Method to find product of three largest distinct elements
public static int productOfThreeLargestDistinct(int[] arr) {

// Max Heap using Priority Queue
```

```
PriorityQueue<Integer> maxHeap =
new PriorityQueue<>(Collections.reverseOrder());

// Set to store distinct elements
Set<Integer> distinctSet = new HashSet<>();

// Insert distinct elements into heap
for (int num : arr) {
if (!distinctSet.contains(num)) {
distinctSet.add(num);
maxHeap.add(num);
}
}

// If less than 3 distinct elements exist
if (maxHeap.size() < 3) {
System.out.println("Not enough distinct elements");
return -1;
}

int product = 1;

// Remove top 3 largest elements
for (int i = 0; i < 3; i++) {
product *= maxHeap.poll();
}

return product;
}
```

```
public static void main(String[] args) {

int[] arr = {10, 3, 5, 6, 20, 20};

int result = productOfThreeLargestDistinct(arr);

if (result != -1) {
System.out.println("Product of three largest distinct elements: " + result);
}
 }
}
```

OUTPUT:

```
Product of three largest distinct elements: 1200


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-07:

AIM : Write a Program to Merge two linked lists(sorted).

Problem Statement:

Given two sorted linked lists, merge them into a single sorted linked list. The new linked list should be created by splicing together the nodes of the first two lists.

Algorithm:

1. **Initialize Pointers**: Start with two pointers, one for each linked list, and a dummy node to build the merged list.
2. **Compare Nodes**: Compare the values of the nodes pointed to by the two pointers.
3. **Append Smaller Node**: Append the smaller node to the merged list and move the corresponding pointer to the next node.
4. **Repeat**: Continue the process until one of the lists is exhausted.
5. **Append Remaining Nodes**: Append the remaining nodes from the non-exhausted list to the merged list.
6. **Return Merged List**: Return the merged list, starting from the node after the dummy node.

CODE:

```
import java.util.Scanner;

// Definition for singly-linked list
class ListNode {
int val;
ListNode next;

ListNode() {}

ListNode(int val) {
this.val = val;
}

ListNode(int val, ListNode next) {
this.val = val;
this.next = next;
}
}
```

```
public class Main {   // The class name must be Main for online compilers

// Merge two sorted linked lists
public static ListNode mergeTwoLists(ListNode list1, ListNode list2) {

ListNode dummy = new ListNode(0);
ListNode tail = dummy;

while (list1 != null && list2 != null) {

if (list1.val <= list2.val) {
tail.next = list1;
list1 = list1.next;
}
else {
tail.next = list2;
list2 = list2.next;
}

tail = tail.next;
}

tail.next = (list1 != null) ? list1 : list2;

return dummy.next;
}

// Helper method to print a linked list
public static void printList(ListNode head) {

ListNode current = head;

while(current != null) {
System.out.print(current.val + " ");
current = current.next;
}

System.out.println();
}

// Helper method to create a linked list from an array
public static ListNode createList(int[] arr) {

if (arr.length == 0) return null;

ListNode head = new ListNode(arr[0]);
ListNode current = head;
```

```
for (int i = 1; i < arr.length; i++) {
current.next = new ListNode(arr[i]);
current = current.next;
}

return head;
}

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

// Input for first linked list
System.out.print("Enter number of elements in List 1: ");
int n1 = sc.nextInt();

int[] arr1 = new int[n1];

System.out.println("Enter elements of List 1 in sorted order:");
for (int i = 0; i < n1; i++) {
arr1[i] = sc.nextInt();
}

ListNode list1 = createList(arr1);

// Input for second linked list
System.out.print("Enter number of elements in List 2: ");
int n2 = sc.nextInt();

int[] arr2 = new int[n2];

System.out.println("Enter elements of List 2 in sorted order:");
for (int i = 0; i < n2; i++) {
arr2[i] = sc.nextInt();
}

ListNode list2 = createList(arr2);

// Merge the lists
ListNode mergedList = mergeTwoLists(list1, list2);

// Print merged list
System.out.print("Merged List: ");
printList(mergedList);

sc.close();
}
}
```

OUTPUT:

```
Enter number of elements in List 1: 5
Enter elements of List 1 in sorted order:
10
30
50
70
90
Enter number of elements in List 2: 5
Enter elements of List 2 in sorted order:
20
40
80
100
120
Merged List: 10 20 30 40 50 70 80 90 100 120

=== Code Execution Successful ===
```

PRACTICAL-08:

AIM : Write a Program to find the Merge point of two linked lists(sorted).

Problem Statement:

Given two sorted linked lists, find the merge point where the two lists merge into a single linked list. The merge point is the first node where the two lists share the same node. If the two lists do not merge, return None.

Algorithm:

1. **Initialize Pointers**: Start with two pointers, one for each linked list (let's call them p1 and p2).
2. **Traverse the Lists**: Traverse both linked lists simultaneously:
   - If the current node in p1 is smaller than the current node in p2, move p1 to the next node.
   - If the current node in p2 is smaller than the current node in p1, move p2 to the next node.
   - If the current nodes in p1 and p2 are equal, that node is the merge point.
3. **Handle Unequal Lengths**: If one list is longer than the other, continue traversing the longer list until the pointers meet.
4. **Return the Merge Point**: If a merge point is found, return the node. If the pointers reach the end of both lists without finding a merge point, return None.

CODE:

```
class ListNode {
int val;
ListNode next;

ListNode(int val) {
this.val = val;
this.next = null;
}
}

public class Solution {

public static ListNode getIntersectionNode(ListNode headA, ListNode headB) {
```

```
// Two pointers
ListNode p1 = headA;


ListNode p2 = headB;

// Traverse until both pointers meet
while (p1 != p2) {

// Move to next node or switch to other list
p1 = (p1 == null) ? headB : p1.next;
p2 = (p2 == null) ? headA : p2.next;
}

// Either intersection node or null
return p1;
}

// Main function
public static void main(String[] args) {

// Create common (intersection) nodes
ListNode common = new ListNode(8);
common.next = new ListNode(10);

// First linked list: 3 -> 7 -> 8 -> 10
ListNode headA = new ListNode(3);
headA.next = new ListNode(7);
headA.next.next = common;

// Second linked list: 99 -> 1 -> 8 -> 10
ListNode headB = new ListNode(99);
headB.next = new ListNode(1);
headB.next.next = common;

// Call function
ListNode intersection = getIntersectionNode(headA, headB);

// Print result
if (intersection != null) {
System.out.println("Intersection Node Value: " + intersection.val);
}
else {
System.out.println("No Intersection");
}
}
}
```

OUTPUT:

```
Intersection Node Value: 8


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-09:

AIM : Write a Program to Swap Nodes pairwise.

Problem Statement:

Given a singly linked list, swap every two adjacent nodes and return the head of the modified list. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Algorithm:

1. **Initialize Pointers:**
   - Use a dummy node to simplify edge cases (e.g., swapping the first two nodes). o Use prev to keep track of the node before the pair to be swapped.
   - Use curr to point to the first node of the pair.
2. **Swap Nodes:**
   - While curr and curr.next are not None:
     - Identify the two nodes to be swapped: first and second.
     - Update prev.next to point to second.
     - Update first.next to point to second.next.
     - Update second.next to point to first.
     - Move prev to first and curr to first.next.
3. **Return the Modified List:**
   - The new head of the list is dummy.next.

CODE:

```
class ListNode {
int val;
ListNode next;

ListNode() {}

ListNode(int val) {
this.val = val;
this.next = null;
}
```

```
ListNode(int val, ListNode next) {


this.val = val;
this.next = next;
}
}

public class Solution {

public static ListNode swapPairs(ListNode head) {

ListNode dummy = new ListNode(0);
dummy.next = head;

ListNode prev = dummy;

while (prev.next != null && prev.next.next != null) {

ListNode first = prev.next;
ListNode second = first.next;

// swapping
first.next = second.next;
second.next = first;
prev.next = second;

prev = first;
}

return dummy.next;
}

// Main method for online compiler
public static void main(String[] args) {

// Create linked list: 1 -> 2 -> 3 -> 4
ListNode head = new ListNode(1);
head.next = new ListNode(2);
head.next.next = new ListNode(3);
head.next.next.next = new ListNode(4);

// Call swapPairs
head = swapPairs(head);

// Print result
System.out.print("After swapping pairs: ");
```

```
while (head != null) {
System.out.print(head.val + " ");
head = head.next;
}
}
}
```

OUTPUT:

```
After swapping pairs: 2 1 4 3

...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-10:

AIM : Write a Program for Building a Function ISVALID to VALIDATE BST.

Problem Statement:

Given a binary tree, write a function isValidBST to determine if it is a valid Binary Search Tree (BST). A BST is valid if:

1. The left subtree of a node contains only nodes with values less than the node's value.
2. The right subtree of a node contains only nodes with values greater than the node's value.
3. Both the left and right subtrees must also be valid BSTs.

Algorithm:

1. **Define a helper function**:
   - The helper function will recursively check each node in the tree.
   - It will take three parameters: the current node, the minimum allowed value (min_val), and the maximum allowed value (max_val).
2. **Base Case**:
   - If the current node is None, return True (an empty tree is a valid BST).
3. **Check Current Node**:
   - If the current node's value is less than or equal to min_val or greater than or equal to max_val, return False (it violates BST rules).
4. **Recursive Case**:
   - Recursively check the left subtree, updating the max_val to the current node's value.
   - Recursively check the right subtree, updating the min_val to the current node's value.
5. **Return Result**:
   - If both subtrees are valid, return True.

CODE:

```
class TreeNode {
int val;
TreeNode left;
TreeNode right;

TreeNode() {}

TreeNode(int val) {
this.val = val;
this.left = this.right = null;
}

TreeNode(int val, TreeNode left, TreeNode right) {
this.val = val;
this.left = left;
this.right = right;
}
}

public class Solution {

// Helper function using min and max range
public static boolean isValidBSTHelper(TreeNode node, long min, long max) {

if (node == null)
return true;

if (node.val <= min || node.val >= max)
return false;

return isValidBSTHelper(node.left, min, node.val) &&
isValidBSTHelper(node.right, node.val, max);
}

// Validate BST
public static boolean isValidBST(TreeNode root) {
return isValidBSTHelper(root, Long.MIN_VALUE, Long.MAX_VALUE);
}

// Main method for online compiler
public static void main(String[] args) {

// Create binary tree
TreeNode root = new TreeNode(8);
root.left = new TreeNode(3);
root.right = new TreeNode(10);
```

```
root.left.left = new TreeNode(1);

root.left.right = new TreeNode(4);   // VALID BST NODE
// root.left.right = new TreeNode(9); // INVALID BST NODE

root.right.right = new TreeNode(14);

// Check if valid BST
if (isValidBST(root)) {
System.out.println("The tree is a Valid BST");
}
else {
System.out.println("The tree is NOT a Valid BST");
}
}
}
```

OUTPUT:

```
The tree is a Valid BST


...Program finished with exit code 0
Press ENTER to exit console.
```

```
The tree is NOT a Valid BST


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-11:

AIM : Write a Program to Build BST.

Problem Statement:

Build a Binary Search Tree (BST):

Given a list of integers, write a program to construct a Binary Search Tree (BST) from the list. The BST should satisfy the following properties:

1. The left subtree of a node contains only nodes with keys less than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. Both the left and right subtrees must also be binary search trees.

Algorithm:

1. **Define the Node Structure:**
   - Each node in the BST will have a value, a left child, and a right child.
2. **Insertion into BST:**
   - Start with the root node. If the tree is empty, the first element becomes the root.
   - For each subsequent element, compare it with the current node:
     - If the element is less than the current node's value, move to the left subtree.
     - If the element is greater than the current node's value, move to the right subtree.
   - Repeat the process until you find an empty spot (where the new node can be inserted).
3. **In-order Traversal (Optional):**
   - To verify the BST, perform an in-order traversal. The output should be a sorted list of elements.

CODE:

```
class TreeNode {
int val;
TreeNode left;
TreeNode right;

TreeNode() {}

TreeNode(int val) {
this.val = val;
this.left = this.right = null;
}

TreeNode(int val, TreeNode left, TreeNode right) {
this.val = val;
this.left = left;
this.right = right;
}
}

public class Solution {

// Insert into BST
public static TreeNode insertIntoBST(TreeNode root, int val) {

if (root == null) {
return new TreeNode(val);
}

if (val < root.val) {
root.left = insertIntoBST(root.left, val);
}
else {
root.right = insertIntoBST(root.right, val);
}

return root;
}

// Inorder traversal to verify BST
public static void inorder(TreeNode root) {

if (root == null)
return;

inorder(root.left);
System.out.print(root.val + " ");
```

```
inorder(root.right);
}

// Main method for online compiler
public static void main(String[] args) {

// Create initial BST
TreeNode root = new TreeNode(4);
root.left = new TreeNode(2);
root.right = new TreeNode(7);
root.left.left = new TreeNode(1);
root.left.right = new TreeNode(3);

// Insert value
int val = 5;
root = insertIntoBST(root, val);

// Print inorder traversal
System.out.print("BST after insertion: ");
inorder(root);
}
}
```

OUTPUT:

```
BST after insertion: 1 2 3 4 5 7

...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-12:

AIM : Write a Program to determine the depth of a given Tree by Implementing MAXDEPTH.

Problem Statement:

Given a binary tree, determine its maximum depth. The maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node. A leaf node is a node with no children.

Algorithm:

1. **Base Case**: If the tree is empty (i.e., the root is None), return 0.
2. **Recursive Case**:
   - Calculate the depth of the left subtree.

     Calculate the depth of the right subtree.
   - The depth of the current node is the maximum of the depths of the left and right subtrees, plus 1 (for the current node).
3. **Return**: The depth of the tree.

CODE:

```
class TreeNode {
int val;
TreeNode left;
TreeNode right;

TreeNode(int val) {
this.val = val;
this.left = null;
this.right = null;
}
}

public class Main {

public int maxDepth(TreeNode root) {

// Base case: empty tree
if (root == null)
return 0;
```

```
// Recursive call
int leftDepth = maxDepth(root.left);
int rightDepth = maxDepth(root.right);

// Return maximum depth
return 1 + Math.max(leftDepth, rightDepth);
}

public static void main(String[] args) {

/*
Example Tree:
1
/ \
2   3
/
4
Depth = 3
*/

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);
root.left.left = new TreeNode(4);

Main sol = new Main();

System.out.println(sol.maxDepth(root)); // Output: 3
}
}
```

OUTPUT:

```
3


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-13:

AIM : Write a Program to Understand and implement Tree traversals i.e. Pre-Order Post Order, In-Order.

Problem Statement:

Implement a binary tree and perform the following tree traversals:

1. **Pre-Order Traversal**: Visit the root node, then traverse the left subtree, and finally traverse the right subtree.
2. **In-Order Traversal**: Traverse the left subtree, visit the root node, and then traverse the right subtree.
3. **Post-Order Traversal**: Traverse the left subtree, traverse the right subtree, and finally visit the root node.

Algorithms:

1. **Pre-Order Traversal**:
   - Visit the root node. ○ Recursively perform a pre-order traversal of the left subtree.
   - Recursively perform a pre-order traversal of the right subtree.
2. **In-Order Traversal**:
   - Recursively perform an in-order traversal of the left subtree.
   - Visit the root node.
   - Recursively perform an in-order traversal of the right subtree.
3. **Post-Order Traversal**:
   - Recursively perform a post-order traversal of the left subtree. Recursively perform a post-order traversal of the right subtree. Visit the root node.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {

// ◈ Inorder Traversal
public static void inorder(TreeNode node, List<Integer> result) {
if (node == null)
return;

inorder(node.left, result);   // Left
result.add(node.val);         // Root
inorder(node.right, result);  // Right
}

// ◈ Preorder Traversal
public static void preorder(TreeNode node, List<Integer> result) {
if (node == null)
return;

result.add(node.val);         // Root
preorder(node.left, result);  // Left
preorder(node.right, result); // Right
}

// ◈ Postorder Traversal
public static void postorder(TreeNode node, List<Integer> result) {
if (node == null)
return;

postorder(node.left, result);  // Left
postorder(node.right, result); // Right
result.add(node.val);          // Root
}

public static void main(String[] args) {
```

```
// Creating sample tree
//       1
//      / \
//     2   3
//    / \
//   4   5

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);
root.left.left = new TreeNode(4);
root.left.right = new TreeNode(5);

List<Integer> inorderResult = new ArrayList<>();
List<Integer> preorderResult = new ArrayList<>();
List<Integer> postorderResult = new ArrayList<>();

inorder(root, inorderResult);
preorder(root, preorderResult);
postorder(root, postorderResult);

System.out.println("Inorder Traversal: " + inorderResult);
System.out.println("Preorder Traversal: " + preorderResult);
System.out.println("Postorder Traversal: " + postorderResult);
}
}
```

OUTPUT:

```
Inorder Traversal: [4, 2, 5, 1, 3]
Preorder Traversal: [1, 2, 4, 5, 3]
Postorder Traversal: [4, 5, 2, 3, 1]


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-14:

AIM : Write a Program to perform Boundary Traversal on BST.

Problem Statement:

Given a Binary Search Tree (BST), perform a **Boundary Traversal** of the tree. Boundary Traversal involves traversing the boundary nodes of the tree in the following order:

1. **Left Boundary**: Traverse the left boundary from the root to the leftmost leaf (excluding the leaf if it has a right subtree).
2. **Leaf Nodes**: Traverse all the leaf nodes from left to right.
3. **Right Boundary**: Traverse the right boundary from the rightmost leaf to the root (excluding the leaf if it has a left subtree).

The traversal should output the nodes in the correct order.

Algorithm:

1. **Left Boundary Traversal**:
   - Start from the root and move to the left child until a leaf node is reached.
   - If a node has no left child, move to the right child.
   - Exclude the leaf node if it has a right subtree.
2. **Leaf Nodes Traversal**:
   - Perform an inorder traversal and collect all leaf nodes.
3. **Right Boundary Traversal**:
   - Start from the root and move to the right child until a leaf node is reached.
   - If a node has no right child, move to the left child.
   - Exclude the leaf node if it has a left subtree.
   - Reverse the collected nodes to get the correct order.
4. **Combine Results**:
   - Combine the left boundary, leaf nodes, and right boundary to get the final result.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {

// Function to check if node is a leaf
static boolean isLeaf(TreeNode node) {
return node.left == null && node.right == null;
}

// Add Left Boundary
static void addLeftBoundary(TreeNode node, List<Integer> result) {
TreeNode curr = node.left;

while (curr != null) {
if (!isLeaf(curr)) {
result.add(curr.val);
}

if (curr.left != null)
curr = curr.left;
else
curr = curr.right;
}
}

// Add Right Boundary
static void addRightBoundary(TreeNode node, List<Integer> result) {
Stack<Integer> stack = new Stack<>();
TreeNode curr = node.right;

while (curr != null) {
if (!isLeaf(curr)) {
stack.push(curr.val);
}
```

```
if (curr.right != null)
curr = curr.right;
else
curr = curr.left;
}

while (!stack.isEmpty()) {
result.add(stack.pop());
}
}

// Add Leaf Nodes
static void addLeaves(TreeNode node, List<Integer> result) {
if (node == null)
return;

if (isLeaf(node)) {
result.add(node.val);
return;
}

addLeaves(node.left, result);
addLeaves(node.right, result);
}

// Boundary Traversal
static List<Integer> boundaryTraversal(TreeNode root) {
List<Integer> result = new ArrayList<>();

if (root == null)
return result;

if (!isLeaf(root))
result.add(root.val);

addLeftBoundary(root, result);
addLeaves(root, result);
addRightBoundary(root, result);

return result;
}

public static void main(String[] args) {
```

```
// Creating Sample Tree
//       20
//      / \
//    10   30
//   / \    \
//  5  15    40

TreeNode root = new TreeNode(20);
root.left = new TreeNode(10);
root.right = new TreeNode(30);
root.left.left = new TreeNode(5);
root.left.right = new TreeNode(15);
root.right.right = new TreeNode(40);

List<Integer> result = boundaryTraversal(root);

System.out.println("Boundary Traversal: " + result);
}
}
```

OUTPUT:

```
Boundary Traversal: [20, 10, 5, 15, 40, 30]


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-15:

AIM : Write a program for Lowest Common Ancestors.

Problem Statement:

Given a binary tree and two nodes p and q, find the lowest common ancestor (LCA) of the two nodes. The lowest common ancestor is the lowest node in the tree that has both p and q as descendants (where we allow a node to be a descendant of itself).

Algorithm:

1. **Base Case:** If the current node is None, return None.
2. **Recursive Case:**
   - If the current node is either p or q, return the current node.
   - Recursively find the LCA in the left subtree.
   - Recursively find the LCA in the right subtree.
3. **Decision:**
   - If both the left and right subtrees return a non-None value, it means p and q are found in different subtrees, so the current node is the LCA.
   - If only one subtree returns a non-None value, return that value (either p or q is found in that subtree).
   - If both subtrees return None, return None.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {
```

```
// LCA Function
public static TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {

if (root == null || root == p || root == q) {
return root;
}

TreeNode left = lowestCommonAncestor(root.left, p, q);
TreeNode right = lowestCommonAncestor(root.right, p, q);

if (left != null && right != null) {
return root;
}

return (left != null) ? left : right;
}

public static void main(String[] args) {

// Creating Sample Tree
//       3
//      / \
//     5   1
//    / \ / \
//   6  2 0  8

TreeNode root = new TreeNode(3);
root.left = new TreeNode(5);
root.right = new TreeNode(1);

root.left.left = new TreeNode(6);
root.left.right = new TreeNode(2);

root.right.left = new TreeNode(0);
root.right.right = new TreeNode(8);

TreeNode p = root.left;   // Node 5
TreeNode q = root.right;  // Node 1

TreeNode lca = lowestCommonAncestor(root, p, q);

System.out.println("Lowest Common Ancestor: " + lca.val);
}
}
```

OUTPUT:

```
Lowest Common Ancestor: 3


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-16:

AIM : Write a Program to verify and validate mirrored trees or not.

Problem Statement:

Given two binary trees, write a program to determine if they are mirrors of each other. Two binary trees are considered mirrors if they are symmetric in structure, and the corresponding nodes have the same values but are mirrored in position.

Algorithm:

1. **Base Case**: If both trees are empty (None), they are mirrors of each other.
2. **Recursive Case**:
   - Check if the current nodes of both trees have the same value.
   - Recursively check if the left subtree of the first tree is a mirror of the right subtree of the second tree.
   - Recursively check if the right subtree of the first tree is a mirror of the left subtree of the second tree.
3. If any of the above conditions fail, the trees are not mirrors.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {

// Function to check if tree is symmetric
```

```
public static boolean isSymmetric(TreeNode root) {
if (root == null)
return true;

return isMirror(root.left, root.right);
}

// Mirror checking function
private static boolean isMirror(TreeNode t1, TreeNode t2) {

if (t1 == null && t2 == null)
return true;

if (t1 == null || t2 == null)
return false;

return (t1.val == t2.val)
&& isMirror(t1.left, t2.right)
&& isMirror(t1.right, t2.left);
}

public static void main(String[] args) {

// Creating Sample Symmetric Tree
//       1
//      / \
//     2   2
//    / \ / \
//   3  4 4  3

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(2);

root.left.left = new TreeNode(3);
root.left.right = new TreeNode(4);

root.right.left = new TreeNode(4);
root.right.right = new TreeNode(3);

boolean result = isSymmetric(root);

if (result)
System.out.println("Tree is Mirror (Symmetric)");
else
System.out.println("Tree is Not Mirror");
}
}
```

OUTPUT:

```
Tree is Mirror (Symmetric)


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-17:

AIM : Write a Program for a basic hash function in a programming language of your choice. Demonstrate its usage to store and retrieve key-value pairs.

Problem Statement:

Design and implement a basic hash function in Python that can be used to store and retrieve key-value pairs. The hash function should map keys to indices in a fixed-size array (hash table). Demonstrate the usage of this hash function by storing and retrieving key-value pairs.

Algorithm:

1. **Define a Hash Function**:
   - The hash function will take a key as input and return an integer representing the index in the hash table.
   - A simple hash function can be implemented by summing the ASCII values of the characters in the key and then taking the modulo with the size of the hash table.
2. **Create a Hash Table**:
   - The hash table will be a list of a fixed size, where each index can store a key- value pair.
3. **Insert Key-Value Pair**:
   - Use the hash function to determine the index for the key.
   - Store the key-value pair at the computed index in the hash table.
4. **Retrieve Value by Key**:
   - Use the hash function to determine the index for the key.
   - Retrieve the value stored at the computed index in the hash table.
5. **Handle Collisions**:
   - For simplicity, this implementation will not handle collisions. In a real-world scenario, techniques like chaining or open addressing would be used.

CODE:

```
import java.util.*;

class MyHashMap {

private int SIZE = 10;
private LinkedList<Entry>[] table;

// Entry class to store key-value pairs
static class Entry {
int key;
int value;

Entry(int key, int value) {
this.key = key;
this.value = value;
}
}

// Constructor
MyHashMap() {
table = new LinkedList[SIZE];
for (int i = 0; i < SIZE; i++) {
table[i] = new LinkedList<>();
}
}

// Basic Hash Function
private int hash(int key) {
return key % SIZE;
}

// Put (store key-value pair)
public void put(int key, int value) {
int index = hash(key);

for (Entry e : table[index]) {
if (e.key == key) {
e.value = value;
return;
}
}

table[index].add(new Entry(key, value));
}

// Get value using key
public int get(int key) {
int index = hash(key);
```

```
for (Entry e : table[index]) {
if (e.key == key) {
return e.value;
}
}

return -1;
}

// Remove key
public void remove(int key) {
int index = hash(key);

table[index].removeIf(e -> e.key == key);
}
}

public class Main {
public static void main(String[] args) {

MyHashMap map = new MyHashMap();

// Store key-value pairs
map.put(1, 100);
map.put(2, 200);
map.put(3, 300);

// Retrieve values
System.out.println("Value for key 1: " + map.get(1));
System.out.println("Value for key 2: " + map.get(2));

// Remove a key
map.remove(2);

System.out.println("Value for key 2 after removal: " + map.get(2));
}
}
```

OUTPUT:

```
Value for key 1: 100
Value for key 2: 200
Value for key 2 after removal: -1


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-18:

AIM : Implement a hash table using separate chaining for collision handling. Perform operations like insertion, deletion, and search on the hash table.

Problem Statement:

Implement a hash table using separate chaining for collision handling. The hash table should support the following operations:

1. **Insertion**: Insert a key-value pair into the hash table. If the key already exists, update the value.
2. **Deletion**: Remove a key-value pair from the hash table based on the key.
3. **Search**: Retrieve the value associated with a given key from the hash table.

Algorithm:

1. **Hash Function**:
   - Use a simple hash function, such as hash(key) % size, where size is the number of buckets in the hash table.
2. **Insertion**:
   - Compute the hash value for the key.
   - Locate the bucket corresponding to the hash value.
   - If the bucket is empty, create a new linked list node with the key-value pair and add it to the bucket.
   - If the bucket is not empty, traverse the linked list to check if the key already exists. If it does, update the value. If not, append the new key-value pair to the end of the linked list.
3. **Deletion**:
   - Compute the hash value for the key.
   - Locate the bucket corresponding to the hash value.
   - Traverse the linked list to find the key. If found, remove the node from the linked list.
4. **Search**:
   - Compute the hash value for the key.
   - Locate the bucket corresponding to the hash value.
   - Traverse the linked list to find the key. If found, return the associated value. If not found, return None.

CODE:

```
import java.util.*;
class HashTable {

// Size of hash table
int SIZE = 10;
// Node class
static class Node {
int key;
int value;
Node next;

Node(int key, int value) {
this.key = key;
this.value = value;
}
}

Node[] table;

// Constructor
HashTable() {
table = new Node[SIZE];
}
// Hash function
int hash(int key) {
return key % SIZE;
}

// Insert operation
void insert(int key, int value) {
int index = hash(key);

Node newNode = new Node(key, value);
newNode.next = table[index];
table[index] = newNode;

System.out.println("Inserted: (" + key + "," + value + ")");
}

// Search operation
void search(int key) {
int index = hash(key);
Node temp = table[index];

while (temp != null) {
if (temp.key == key) {
System.out.println("Key " + key + " found with value " + temp.value);
```

```
}
temp = temp.next;
}

System.out.println("Key " + key + " not found");
}
// Delete operation
void delete(int key) {
int index = hash(key);
Node temp = table[index];
Node prev = null;

while (temp != null) {
if (temp.key == key) {
if (prev == null)
table[index] = temp.next;
else
prev.next = temp.next;

System.out.println
prev = temp;
temp = temp.next;
}

System.out.println("Key " + key + " not found for deletion");
}
}
public class Main {

public static void main(String[] args) {

HashTable ht = new HashTable();

// Insert elements
ht.insert(10, 100);
ht.insert(20, 200);
ht.insert(15, 150);
ht.insert(25, 250);

// Search element
ht.search(15);

// Delete element
ht.delete(20);

// Search again
ht.search(20);
}
}
```

OUTPUT:

```
Inserted: (10,100)
Inserted: (20,200)
Inserted: (15,150)
Inserted: (25,250)
Key 15 found with value 150
Key 20 deleted
Key 20 not found


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-19:

AIM : Write a Program to Implement Two sums using HASHMAP.

Problem Statement:

Given an array of integers nums and an integer target, return the indices of the two numbers such that they add up to the target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order.

Algorithm:

1. **Initialize a Hash Map**: Create an empty hash map (dictionary in Python) to store the numbers and their indices.
2. **Iterate through the array**: Loop through the array nums using an index i.
3. **Calculate the complement**: For each number nums[i], calculate the complement as complement = target - nums[i].
4. **Check if the complement exists in the hash map**:
   - If it exists, return the current index i and the index stored in the hash map for the complement.
   - If it does not exist, store the current number nums[i] and its index i in the hash map.
5. **Return the result**: If no solution is found, return an empty list or handle it as per requirements.

CODE:

```
import java.util.*;

public class Main {

// Function to find two sum using HashMap
public static int[] twoSum(int[] nums, int target) {

HashMap<Integer, Integer> map = new HashMap<>();

for (int i = 0; i < nums.length; i++) {

int complement = target - nums[i];

if (map.containsKey(complement)) {
return new int[]{map.get(complement), i};
}

map.put(nums[i], i);
}
```

```
return new int[]{-1, -1};
}

public static void main(String[] args) {

int[] nums = {2, 7, 11, 15};
int target = 9;

int[] result = twoSum(nums, target);

System.out.println("Indices of Two Sum: " + result[0] + " , " + result[1]);
}
}
```

OUTPUT:

```
Indices of Two Sum: 0 , 1


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-20:

AIM : Write a Program to Implement Search, insert, and Remove in Trie.

Problem Statement:

A **Trie** (pronounced "try") is a tree-like data structure used to store a dynamic set of strings. It is particularly useful for operations like searching, inserting, and removing strings efficiently. Each node in a Trie represents a character of a string, and the path from the root to a node represents a prefix of one or more strings.

Your task is to implement a Trie data structure in Python with the following operations:

1. **Insert**: Insert a string into the Trie.
2. **Search**: Check if a string exists in the Trie.
3. **Remove**: Remove a string from the Trie.

Algorithm:

1. **Trie Node Structure**:
   - Each node in the Trie will have:
     - A dictionary to store child nodes (key: character, value: child node).
     - A boolean flag is_end_of_word to indicate if the node represents the end of a word.
2. **Insert Operation**:
   - Start from the root node.
   - For each character in the string, check if it exists in the current node's children.
   - If it doesn't exist, create a new node and add it to the children.
   - Move to the child node and repeat the process for the next character.
   - After processing all characters, mark the last node as the end of the word.
3. **Search Operation**:
   - Start from the root node.
   - For each character in the string, check if it exists in the current node's children.
   - If it doesn't exist, return False.
   - Move to the child node and repeat the process for the next character.
   - After processing all characters, check if the last node is marked as the end of a word.
4. **Remove Operation**:
   - Start from the root node.

- For each character in the string, check if it exists in the current node's children.
- If it doesn't exist, return False (string not found).
- Move to the child node and repeat the process for the next character.
- After processing all characters, mark the last node as not being the end of a word.
- If the node has no children, remove it from its parent's children.

CODE:

```
import java.util.*;

class TrieNode {
TrieNode[] children = new TrieNode[26];
boolean isEndOfWord;
}

class Trie {

private TrieNode root;

public Trie() {
root = new TrieNode();
}

// Insert word into Trie
public void insert(String word) {
TrieNode node = root;

for (char c : word.toCharArray()) {
int index = c - 'a';

if (node.children[index] == null) {
node.children[index] = new TrieNode();
}

node = node.children[index];
}

node.isEndOfWord = true;
System.out.println("Inserted: " + word);
}
```

```
// Search word in Trie
public boolean search(String word) {
TrieNode node = root;

for (char c : word.toCharArray()) {
int index = c - 'a';

if (node.children[index] == null) {
return false;
}

node = node.children[index];
}

return node.isEndOfWord;
}

// Remove word from Trie
public void remove(String word) {
if (search(word)) {
delete(root, word, 0);
System.out.println("Deleted: " + word);
} else {
System.out.println("Word not found: " + word);
}
}

private boolean delete(TrieNode node, String word, int depth) {
if (node == null)
return false;

if (depth == word.length()) {
if (node.isEndOfWord)
node.isEndOfWord = false;

for (TrieNode child : node.children)
if (child != null)
return false;

return true;
}
```

```
int index = word.charAt(depth) - 'a';

if (delete(node.children[index], word, depth + 1)) {
node.children[index] = null;

return !node.isEndOfWord && isEmpty(node);
}

return false;
}
private boolean isEmpty(TrieNode node) {
for (TrieNode child : node.children)
if (child != null)
return false;

return true;
}
}

public class Main {
public static void main(String[] args) {

Trie trie = new Trie();

// Insert words
trie.insert("apple");
trie.insert("app");
trie.insert("bat");

// Search words
System.out.println("Search apple: " + trie.search("apple"));
System.out.println("Search app: " + trie.search("app"));
System.out.println("Search bat: " + trie.search("bat"));
System.out.println("Search ball: " + trie.search("ball"));

// Remove word
trie.remove("app");
// Search again
System.out.println("Search app after deletion: " + trie.search("app"));
}
}
```

OUTPUT:

```
Inserted: apple
Inserted: app
Inserted: bat
Search apple: true
Search app: true
Search bat: true
Search ball: false
Deleted: app
Search app after deletion: false


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-21:

AIM : Write a Program to Implement Huffman coding.

Problem Statement:

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, with shorter codes assigned to more frequent characters. The algorithm builds a binary tree (Huffman tree) based on the frequency of characters and generates codes by traversing the tree. The goal is to implement Huffman coding to compress and decompress a given input string.

Algorithm:

1. **Frequency Calculation**: Calculate the frequency of each character in the input string.
2. **Priority Queue**: Create a priority queue (min-heap) where the node with the lowest frequency has the highest priority.
3. **Build Huffman Tree**:
   - Extract two nodes with the lowest frequency from the priority queue.
   - Create a new internal node with a frequency equal to the sum of the two nodes' frequencies.
   - Add the new node back to the priority queue.
   - Repeat until there is only one node left in the queue, which becomes the root of the Huffman tree.
4. **Generate Codes**: Traverse the Huffman tree to assign binary codes to each character.
5. **Encode**: Use the generated codes to encode the input string.
6. **Decode**: Use the Huffman tree to decode the encoded string back to the original string.

CODE:

```
import java.util.*;

// Node class
class Node {
char ch;
int freq;
Node left, right;

Node(char ch, int freq) {
this.ch = ch;
this.freq = freq;
left = right = null;
```

```
}
}

// Comparator for priority queue
class MyComparator implements Comparator<Node> {
public int compare(Node x, Node y) {
return x.freq - y.freq;
}
}

public class Main {

// Function to print Huffman Codes
public static void printCode(Node root, String s) {

if (root.left == null && root.right == null) {
System.out.println(root.ch + " : " + s);
return;
}

printCode(root.left, s + "0");
printCode(root.right, s + "1");
}

public static void main(String[] args) {

char[] charArray = { 'a', 'b', 'c', 'd', 'e', 'f' };
int[] charFreq = { 5, 9, 12, 13, 16, 45 };

PriorityQueue<Node> q = new PriorityQueue<>(new MyComparator());

// Create leaf nodes
for (int i = 0; i < charArray.length; i++) {
q.add(new Node(charArray[i], charFreq[i]));
}

Node root = null;

// Build Huffman Tree
while (q.size() > 1) {
```

```
Node x = q.poll();
Node y = q.poll();

Node f = new Node('-', x.freq + y.freq);
f.left = x;
f.right = y;

root = f;
q.add(f);
}

System.out.println("Huffman Codes:");
printCode(root, "");
}
}
```

OUTPUT:

```
Huffman Codes:
f : 0
c : 100
d : 101
a : 1100
b : 1101
e : 111


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-22:

AIM : Write a Program to find Distinct substrings in a string.

Problem Statement:

Given a string, find the number of distinct substrings in the string. A substring is a contiguous sequence of characters within a string. For example, the string "abc" has the following substrings: "a", "b", "c", "ab", "bc", and "abc". The total number of distinct substrings is 6.

Algorithm:

1. **Initialize a set to store unique substrings**: A set is used because it automatically handles duplicates.
2. **Iterate over the string**: Use nested loops to generate all possible substrings.
   - The outer loop runs from the start of the string to the end.
   - The inner loop runs from the current position of the outer loop to the end of the string.
3. **Generate substrings**: For each pair of indices (i, j), extract the substring from index i to j+1.
4. **Store substrings in the set**: Add each generated substring to the set.
5. **Count the number of unique substrings**: The size of the set at the end of the iteration will be the number of distinct substrings.
6. **Return the count**: Output the number of distinct substrings.

CODE:

```
import java.util.*;

public class Main {

// Function to find distinct substrings
public static int countDistinctSubstrings(String s) {

HashSet<String> set = new HashSet<>();

for (int i = 0; i < s.length(); i++) {

String substring = "";
```

```
for (int j = i; j < s.length(); j++) {
substring += s.charAt(j);
set.add(substring);
}
}

return set.size();
}

public static void main(String[] args) {

String str = "ababa";

int result = countDistinctSubstrings(str);

System.out.println("Total Distinct Substrings: " + result);
}
}
```

OUTPUT:

```
Total Distinct Substrings: 9


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-23:

AIM : Write a Program to find The No of Words in a Trie.

Problem Statement

Given a tree data structure, where each node contains a word, write a program to find the total number of words in the tree. The tree can be represented as a collection of nodes, where each node has a value (the word) and a list of children nodes.

Algorithm

1. Define the Tree Node Structure:
   - Each node should have a value (the word) and a list of children nodes.
2. Traverse the Tree:
   - Use a Depth-First Search (DFS) or Breadth-First Search (BFS) approach to traverse the tree.
   - For each node visited, increment a counter to keep track of the number of words.
3. Return the Count:
   - After traversing the entire tree, return the total count of words.

CODE:

```
import java.util.*;
class TrieNode {
TrieNode[] children = new TrieNode[26];
boolean isEnd;
}

class Trie {
TrieNode root;

Trie() {
root = new TrieNode();
}
// Insert word
void insert(String word) {
TrieNode node = root;
for (char c : word.toCharArray()) {
int index = c - 'a';
if (node.children[index] == null) {
node.children[index] = new TrieNode();
}
```

```
node = node.children[index];
}

node.isEnd = true;
}

// Count words in Trie
int countWords(TrieNode node) {
if (node == null)
return 0;

int count = 0;

if (node.isEnd)
count++;

for (int i = 0; i < 26; i++) {
count += countWords(node.children[i]);
}
return count;
}

int getWordCount() {
return countWords(root);
}
}

public class Main {
public static void main(String[] args) {
Trie trie = new Trie();

// Insert words
trie.insert("apple");
trie.insert("app");
trie.insert("bat");
trie.insert("ball");

// Count total words
System.out.println("Total words in Trie: " + trie.getWordCount());
}
}
```

OUTPUT:

```
Total words in Trie: 4


...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-24:

AIM : Write a Program to view a tree from left View.

Problem Statement:

Given a binary tree, print the left view of the tree. The left view of a binary tree contains all nodes that are visible when the tree is viewed from the left side. The leftmost node at each level is included in the left view.

Algorithm:

1. If the tree is empty, return an empty list.
2. Use a queue to perform a level-order traversal of the tree.
3. For each level, store the first node encountered in the left view list.
4. Continue traversing the tree level by level until all nodes are visited.
5. Print or return the left view list.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {

static int maxLevel = 0;

// Function to print left view
public static void leftView(TreeNode node, int level) {

if (node == null)
return;

if (level > maxLevel) {
System.out.print(node.val + " ");
```

```
maxLevel = level;
}

leftView(node.left, level + 1);
leftView(node.right, level + 1);
}

public static void main(String[] args) {

// Creating Sample Tree
//       1
//      / \
//     2   3
//    / \   \
//   4   5   6

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);

root.left.left = new TreeNode(4);
root.left.right = new TreeNode(5);

root.right.right = new TreeNode(6);

System.out.print("Left View of Tree: ");
leftView(root, 1);
}
}
```

OUTPUT:

```
Left View of Tree: 1 2 4

...Program finished with exit code 0
Press ENTER to exit console.
```

PRACTICAL-25:

AIM : Write a Program to Traverse a Tree using Level Order Traversal.

Problem Statement:

Given a binary tree, write a program to perform a **Level Order Traversal** (also known as **Breadth-First Traversal**). This traversal visits all nodes at each depth level before moving on to the next level.

Algorithm:

1. If the tree is empty, return.
2. Initialize a queue and enqueue the root node.
3. While the queue is not empty:
   - Dequeue a node from the front.
   - Process the dequeued node (print or store its value).
   - If the dequeued node has a left child, enqueue it.
   - If the dequeued node has a right child, enqueue it.
4. Repeat until all nodes have been processed.

CODE:

```
import java.util.*;

class TreeNode {
int val;
TreeNode left, right;

TreeNode(int x) {
val = x;
left = right = null;
}
}

public class Main {

// Level Order Traversal Function
public static void levelOrder(TreeNode root) {

if (root == null)
return;
```

```
Queue<TreeNode> queue = new LinkedList<>();
queue.add(root);

while (!queue.isEmpty()) {

TreeNode node = queue.poll();
System.out.print(node.val + " ");

if (node.left != null)
queue.add(node.left);

if (node.right != null)
queue.add(node.right);
}
}

public static void main(String[] args) {

// Creating Sample Tree
//       1
//      / \
//     2   3
//    / \   \
//   4   5   6

TreeNode root = new TreeNode(1);
root.left = new TreeNode(2);
root.right = new TreeNode(3);

root.left.left = new TreeNode(4);
root.left.right = new TreeNode(5);

root.right.right = new TreeNode(6);

System.out.print("Level Order Traversal: ");
levelOrder(root);
}
}
```

OUTPUT:

```
Level Order Traversal: 1 2 3 4 5 6

...Program finished with exit code 0
Press ENTER to exit console.
```